OWASP & Secure Coding

The Attacks Your Code Needs to Survive

ACCTMIS 4630

Week 13 - April 6th, 2026

This Week's Journey

Today	OWASP and Secure Coding
Wednesday	Quality Assurance and Checklist; prep for M5
Friday Lab	M5 workshop

Upcoming

- **Milestone 5:** Due April 13th

Your Attacker Has a Journey Too

Let’s use tools that we know: persona, journey map, design thinking…

Who are potential bad people in our neighbor hood?

- A student wanting to see other people's orders
- A bored CS student poking at your API
- Someone trying to get free products
- A criminal gang trying to steal credit card numbers

Attacker Personas

Attacker Type	Skill Level	Tools	Goal
Script Kiddie	Low	Automated scanners, tutorials	Vandalism, bragging rights
Opportunist	Medium	Browser DevTools, Postman, Burp Suite	Free stuff, data theft
Targeted Attacker	High	Custom scripts, social engineering	Specific data, system control

Key insight: Most attacks exploit well-known vulnerabilities. You don't need to defend against nation-states. You need to close the doors that are already documented.

The Industry Standard for Web Security

OWASP: Open Worldwide Application Security Project

- Non-profit foundation focused on web application security
- Publishes the **Top 10 l**ist of critical web security risks
- Updated periodically (current version: 2025)
- Used by every major company as a security baseline
- Referenced in compliance frameworks (PCI-DSS, SOC 2, HIPAA)
- OWASP also publishes new list for:
  - OWASP for LLMs and Gen AI
  - OWASP for MCP

The OWASP Top 10 (2025)

1. **Broken Access Control** ← We'll focus here
2. **Security Misconfiguration**
3. **Software Supply Chain Failures**
4. **Cryptographic Failures**
5. **Injection** ← We'll focus here
6. **Insecure Design**
7. **Authentication Failures** ← We'll focus here
8. **Software and Data Integrity Failures**
9. **Security Logging and Alerting Failures**
10. **Mishandling of Exceptional Conditions**

Most Critical: Broken Access Control

What it means: Users can act outside their intended permissions.

Real-world examples:

- Changing `/api/orders/123` to `/api/orders/124` and seeing someone else's order
- A regular user accessing `/api/admin/users` by guessing the URL
- Modifying a JWT payload to change `role: "Customer"` to `role: "Admin"`

In Buckeye Marketplace terms:

```
GET /api/orders/5          ← Sarah's order (OK)
GET /api/orders/6          ← Marcus's order (NOT OK — Sarah shouldn't see this)
DELETE /api/products/1     ← Only Admin should do this
```

The test: Can a user access resources that don't belong to them by changing an ID in the URL?

Most Critical: Broken Access Control

What is wrong here?

```
// VULNERABLE — Can you see why?
[Authorize]
[HttpGet("orders/{userId}")]
public async Task<IActionResult> GetOrders(int userId)
{
    var orders = await _context.Orders
        .Where(o => o.UserId == userId)
        .ToListAsync();
    return Ok(orders);
}
```

Most Critical: Broken Access Control

The User Identity comes from the JWT Token, Not the URL

```
// FIXED
[Authorize]
[HttpGet("orders")]
public async Task<IActionResult> GetOrders()
{
    var userId = int.Parse(User.FindFirst(ClaimTypes.NameIdentifier)!.Value);
    var orders = await _context.Orders
        .Where(o => o.UserId == userId)
        .ToListAsync();
    return Ok(orders);
}
```

Injection

What it means: Untrusted data is sent to an interpreter as part of a command or query.

Classic SQL Injection

What the developer intended:

```
SELECT * FROM Products WHERE Name = 'Textbook'
```

What the attacker sends:

```
SELECT * FROM Products WHERE Name = '' OR 1=1; DROP TABLE Products; --'
```

If this statement runs, all of your products in your catalog are **deleted**.

Injection

What it means: Untrusted data is sent to an interpreter as part of a command or query.

Classic SQL Injection

What the developer intended:

```
SELECT * FROM Products WHERE Name = 'Textbook'
```

What the attacker sends:

```
SELECT * FROM Products WHERE Name = '' OR 1=1; DROP TABLE Products; --'
```

If this statement runs, all of your products in your catalog are **deleted**.

Injection

SQL Injection: unprotected data access in API controller

```
// VULNERABLE — string concatenation with user input
[HttpGet("search")]
public async Task<IActionResult> Search(string query)
{
    var sql = $"SELECT * FROM Products WHERE Name LIKE '%{query}%'";
    var products = await _context.Products
        .FromSqlRaw(sql)
        .ToListAsync();
    return Ok(products);
}
```

Attacker sends: query = "'; DROP TABLE Products; --"

Injection

SQL Injection Protection: just use LINQ

```
// SAFE — parameterized query
var products = await _context.Products
    .FromSqlInterpolated($"SELECT * FROM Products WHERE Name LIKE '%' + {query} + '%'")
    .ToListAsync();

// SAFEST — use LINQ (what you should be doing)
var products = await _context.Products
    .Where(p => p.Name.Contains(query))
    .ToListAsync();
```

Injection

Other types of Injection

- Command injection — OS commands via user input
- LDAP injection — directory service queries
- Cross-site scripting (XSS) — injecting JavaScript into web pages (we'll cover this separately)
- **NEW:** Prompt Injection

Authentication Failures

What it means: Weaknesses in authentication allow attackers to assume other users' identities.

Failure	Risk	Your M5 Code
Weak passwords allowed	Brute force attack	Do you enforce complexity?
No rate limiting on login	Automated password guessing	Unlimited login attempts?
Tokens never expire	Stolen token = permanent access	What's your JWT expiration?
Secrets in source code	Anyone with repo access has keys	Is your signing key in `appsettings.json` or User Secrets?
Plain-text password storage	Data breach = all passwords leaked	Are you using `PasswordHasher<T>`?

XSS- Cross-Site Scripting

What it is: Attacker injects JavaScript into your web page that runs in other users' browsers.

Common scenario:

1. Attacker submits a product review containing malicious JavaScript
2. Your app saves it to the database (it's just text, right?)
3. Another user views the product page — the "review" executes as JavaScript
4. The script can act as the user in the page context, and if your app stores auth data in browser-accessible storage it may also be able to read and exfiltrate it

XSS- Cross-Site Scripting

What it is: Attacker injects JavaScript into your web page that runs in other users' browsers.

Types:

1. Stored XSS — malicious script saved in database (most dangerous)
2. Reflected XSS — malicious script in URL parameter, reflected back in response
3. DOM-based XSS — client-side JavaScript processes untrusted data

XSS: React Protections

```
// SAFE — React escapes this automatically
function ProductReview({ text }: { text: string }) {
  return <p>{text}</p>;
}
// Input: "<script>alert('hacked')</script>"
// Renders as: &lt;script&gt;alert('hacked')&lt;/script&gt;
// The script does NOT execute
```

```
// VULNERABLE — dangerouslySetInnerHTML bypasses escaping
function ProductReview({ text }: { text: string }) {
  return <p dangerouslySetInnerHTML={{ __html: text }} />;
}
// Input: "<script>alert('hacked')</script>"
// The script EXECUTES in the user's browser
```

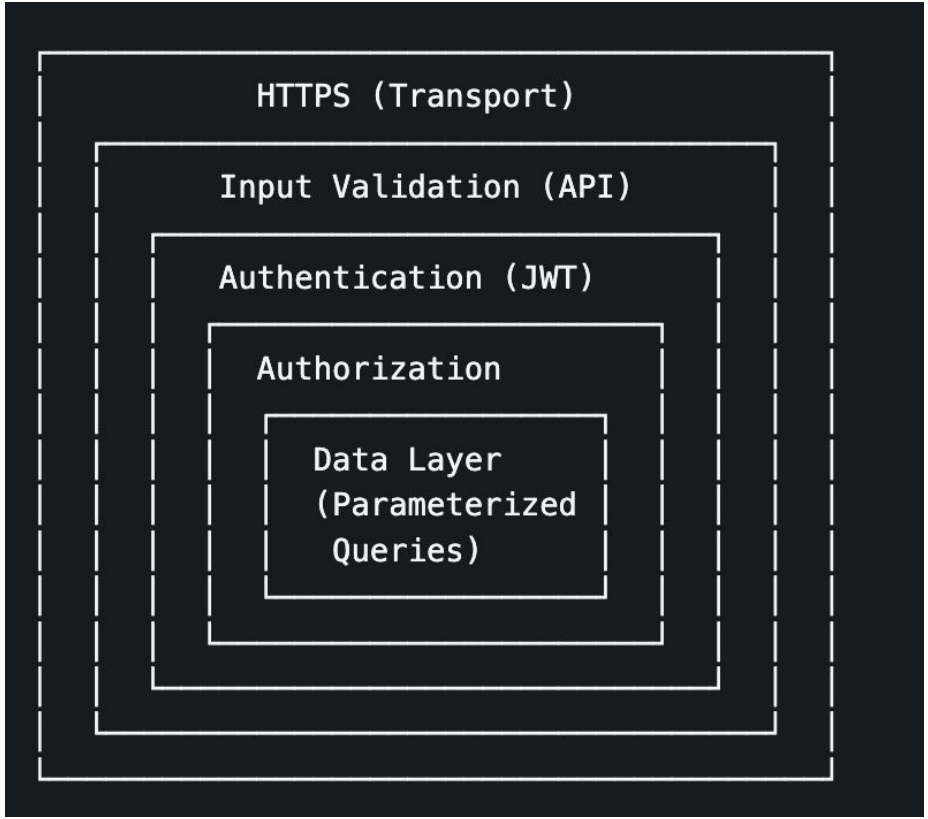
CSRF: Cross-Site Request Forgery

What it is: Attacker tricks your browser into making a request to a site where you're already authenticated

Common scenario:

1. Sarah is logged into Buckeye Marketplace (JWT token in a cookie)
2. Sarah visits a malicious website (e.g., clicks a link in an email)
3. The malicious page contains: <img src="https://marketplace.osu.edu/api/cart/clear" />
4. Sarah's browser makes the request — with her auth cookie attached automatically
5. Sarah's cart is cleared without her knowledge

System Thinking: Defense in Depth



(In)Famous Breeches

Company	Year	Vulnerability	Impact
Equifax	2017	Unpatched third-party component	147M records exposed
Capital One	2019	SSRF + cloud misconfiguration	100M records exposed
SolarWinds	2020	Software supply chain compromise	18,000+ organizations compromised
Log4Shell	2021	Injection in logging library	Affected millions of Java applications
MOVEit	2023	SQL injection	2,700+ organizations, 90M+ individuals